Security Policy Automation Using Large Language Models: A Unified LLM Framework for Policy Synthesis and Verification

Harsh Sharma
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
harshsharma1338@gmail.com

Ganesh Khekare
School of Computer Science
Vellore Institute of Technology
Vellore, India
ganesh.khekare@vit.ac.in

***Abstract*—Manual security policy setup is becoming less useful and more likely to cause mistakes because 5G/6G networks, cloud microservices, and complex System-on-Chip designs are becoming more complicated. A key problem is the difference in meaning between what a company wants to do at a high level and how it's done technically at a low level. This often leads to incorrect setups and security problems. This study introduces a single system for using Large Language Models to automatically handle security policies and help create independent, intent-driven network and system protection. We look at and combine the newest ways to do things, like using closed-loop reinforcement learning for Zero-Touch Networks, making fake data to broaden access control policies, translating intent using tree structures for YANG model mapping, and finding policies in container settings by looking at logs. We analyze algorithms like Security-Aware Group Relative Policy, tree-edit distance mapping, and set-cover to better understand how LLMs can be used for automation. Data shows that LLM systems that are made for a specific area perform better than normal deep learning methods when it comes to finding policies, creating automatic responses, and spotting hardware weaknesses. We also suggest an agent system based on Retrieval-Augmented Generation that can create policies from scratch with reliable approval safeguards.**

***Keywords*—Security Policy Automation, Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), Cybersecurity, Zero-Touch Networks, Intent-Based Networking, Policy Verification, OpenCode Agent, ISO/IEC 27001, Network Security**

I. Introduction

Cybersecurity defenses have always grown along with the systems they defend. Early on, security was fairly basic, admins configured firewall rules and access lists according to their knowledge and experience. Now, things are different. We have fast connections in the form of 5G (and soon 6G) networks, constantly evolving cloud services, and complicated computer chip designs. Because of these shifts, manually setting up security is not practical anymore. We are seeing a move toward automated systems controlled by machines [1], [2], as cyber threats are coming faster and in greater numbers. The main problem now is the semantic gap. This gap between what a business wants to do for security and how it is actually done technically. Any security engineer wants Zero Trust for all devices, but turning that idea into thousands of specific settings on different devices from different companies is very demanding and often leads to mistakes. These mistakes are often the cause of huge security problems because normal templates and standard algorithms cannot handle today's rapidly evolving tech environments. This is where Large Language Models (LLMs) can help us stay secure. Unlike older automation tools that used strict rules, LLMs (e.g., GPT-style and Claude models) can understand natural language well enough to turn instructions into actual security code [1]. This starts a period of Security Policy Automation using LLMs. The process of observing, orienting and deciding can now happen in significantly less time thus, enabling faster automated decision-making cycles in zero-touch networks. This allows us to use Zero-Touch network operations that can automatically adjust and respond when they detect threats.

II. Motivation

This work stems from the demands of future infrastructure. As we move to 6G networks we need things to happen fast. This means security has to react, it cannot wait for a human to intervene. The security systems have to find problems on their own and figure out how to fix those issues [8]. With the widespread adoption of systems like Kubernetes, where workloads are rapidly changing, it is important to find the simplest security rules automatically based on traffic patterns. Previous rules don't work when things are updated at these speeds. Lastly, stronger and persistent attacks mean that we need new ways to check for problems. Regular checks like formal proof and fuzz testing are difficult to scale up as the systems grow. LLMs could help us by making test setups and security checks on their own without needing extensive human intervention, reducing the time spent on verifying hardware designs.

III. Literature Review

The use of Large Language Models in Cybersecurity is changing how security data is analyzed, interpreted, and

created in our world today. The pace at which model architectures for cyber-security related tasks are evolving is clearly illustrated through a number of recent research studies [1], [7]. Traditionally the primary architecture used in most early models was an encoder-based architecture using BERT to assist with classification tasks like identifying phishing email. However, since then, there has been a trend towards using modern decoder-only LLMs (e.g., GPT-style models). Decoder-only models provide additional functionality beyond just classifying a piece of data. They are capable of generating output. The reason for this evolution from encoder-only models is due to the fact that Decoder-only models enable the generation of actionable outputs beyond classification tasks [1], [9]. Encoder-only models were successful in finding/classifying vulnerabilities; however, decoder-only models are able to generate follow-up actions required to address identified vulnerabilities, create scripts necessary to implement corrective action, create policy code, etc. Without human intervention, the ability to automatically generate follow-on actions, and/or automated implementation of previously identified security vulnerabilities and their remedies are critical to Policy Automation [7]. As mentioned earlier, systems must be able to take abstract security concepts and convert them into operationalized solutions such as iptables rule sets or Kubernetes Network Policies. However, the ability to create anything provides opportunities to introduce errors. Errors may exist where models produce acceptable-looking solution configurations that may be inconsistent or contain logical conflicts due to limitations of LLMs [6] that ultimately could be considered either unacceptably risky or unsafe. Therefore, several researchers have begun exploring methods to mitigate these potential errors. Examples include retrieval-augmented generation (RAG), which relies upon existing trusted documentation for the generated output(s); and chain-of-thought (CoT) prompting, which requires that the model's reasoning process occur in a series of discrete steps.

The increasing autonomy of security teams in performing their duties has led to an increase in the use of Security Orchestration, Automation and Response (SOAR) tools. Predefined playbook-based SOAR solutions provide a rigid "if this, then that" structure, and as such work best for repetitive actions with no variance. In addition, predefined playbooks are limited in their ability to respond to unknown or previously unseen threat vectors. A new research area has been developed called Agentic SOAR, and utilizes large language models (LLMs) to allow decision-making capabilities for the agents so they can adjust their responses accordingly. Agents will utilize their current workspace, think through what is needed to be accomplished and run the appropriate tooling necessary to accomplish the task. For instance; when an individual identifies a potential vulnerability within their DevSecOps pipeline, the agent would review the code to identify the source of the issue, create a solution, verify it works properly, and push a pull request to include it into the repository.

IBN has used AI/advanced language model again recently. IBN was designed to simplify network complexity via abstraction with high level intent. However, previous versions have suffered from ambiguity in natural language. The Security Policy Translator [3] represents the state-of-the-art by viewing policy translation as a structural task rather than an abstract task based on keywords or direct machine translations. Using tree-edit distance methods (i.e., Zhang-Shasha), the translator combines abstract user intent represented in YANG CFI with concrete setup of low-level devices in YANG NFI; this enables accurate mapping of structural form and semantic meaning. In addition, such structurally correct and semantically meaningful translation will be critical in order to correctly translate policies in Zero Trust Networks (ZTN) where incorrect interpretation can result in significant loss of service.

In addition to the development of software and network security applications, LLMs are beginning to be used for developing hardware security as well. Specifically, they are being applied primarily in the area of System on Chip (SoC) design [5]. The expense and long time needed to perform checks at the SoC creation phase have made it a major delay in the development of SoCs. Recent research has indicated that LLMs may also serve as automated checking engineers. In order to do this, LLMs will analyze Register Transfer Language (RTL) code which will identify potential vulnerabilities such as finite state machine deadlocks or information flow paths which would normally be missed by other static analysis tools. Furthermore, LLMs are able to support SystemVerilog Assertions (SVAs), to verify the security attributes within the SoC design, thereby moving the security verification from after the fact to before the fact. By doing so, the cost savings associated with performing these verifications during the initial stages of hardware design will result in a much stronger final product.

Container security is an additional area where LLMs are making changes in cloud-native environments. Handling security policy for Kubernetes-based microservice architectures is difficult due to their rapid growth, constant change, and large number of containers. Kunerva [4], which uses a log-driven discovery approach to collect network traffic logs to identify application communication types, provides a behavior-driven approach that creates the fewest possible Kubernetes Network Policies to enable correct communication using a default deny rule. This behavior-driven approach aligns with Zero-Trust principles and illustrates how LLMs can infer work intent from observed system behaviors enabling adaptive and evolving security in emerging cloud environments.

IV. Problem Context

Gap 1: Lack of Datasets. The first constraint is that there simply aren't enough well-organized datasets with sufficient security specificity for training/evaluating a model. General purpose corpora used by many Language Models (LLMs), include a broad base of knowledge about all areas of interest but contain insufficiently detailed domain specific structures and concepts to capture details of Access Control Policy (ACP) or hardware description languages (HDL) [6], [10]. Most available datasets, including those developed from university

policy documents or medical record information, do not have either sufficient size or complexity to model systems like the IoT or critical public infrastructure. As a result of privacy constraints, researchers cannot use real security related data. Therefore they are forced to create synthetic data sets; however it remains to be determined how useful/facilitative these will be.

Gap 2: The challenges of validation and error correction. The challenge is in validating the output of an LLM compared to a known set of security requirements. As LLM's generate text that is plausibly correct they also have the same stochastic (random) nature which could result in producing policy statements that are logically consistent with each other but violate some security requirement. Identifying these types of errors for example two contradictory permissions for one subject within the same policy set are not trivial [6], [10]. Thus far, all methods used to find such contradictions are plagued by excessive false positives and there are no well-defined procedures to validate the correctness and consistency of automated security policies.

Gap 3: Another problem is validating outputs of LLMs for compliance with existing security specifications. Although an LLM output could look "plausible," its random/ stochastic nature may cause it to generate seemingly correct policy rules, but which in fact are flawed (i.e. containing logical errors/security holes). The process of detecting these contradictions (e.g. conflicts in permission levels in policy sets) remains difficult [6]. Currently available validation techniques have a relatively high rate of false positives; this further demonstrates the need for better and more reliable methods for determining the accuracy and coherence of policy created by automated processes.

V. METHODOLOGY

This paper uses a Comparative Prototype-Based approach to evaluate how the use of Retrieval-Augmented Generation (RAG) impacts the overall quality of the security audit report produced by Large Language Models (LLMs) for Technical Configuration Artifacts. Rather than developing a customized model specifically for security tasks this research extended an already existing open source coding agent called OpenCode with a lightweight retrieval mechanism utilizing excerpts from established standards of ISO security controls. The enhanced system was compared to two baselines; one being the direct prompting of a General-Purpose LLM and another as the baseline being the Unmodified OpenCode Agent.

Let us formally represent the input artifact as x for example a Kubernetes YAML file, and y be the audit report generated. In the case of both the baseline agent and direct prompting scenarios, we can formulate the generation as a probability distribution, specifically as $P(y \mid x)$. The proposed method introduces the incorporation of an external knowledge base $D = \{d_1, d_2, \ldots, d_n\}$ to provide relevant snippets $R(x) \subset D$ for each item in the knowledge base to generate (i.e., for each snippet to be an excerpt from ISO security control aligned with standards). For example, when you have provided an input x, a retrieval function identifies which items are likely most relevant among $R(x) \subset D$. Subsequently, generation proceeds according to $P(y \mid x, R(x))$. Therefore, this formulation represents our central hypothesis that utilizing external standards during generation reduces the semantic difference between the low level configuration information and how one interprets those configurations as high-level security controls.

The retrieval stage utilizes a relevance scoring function to compare the query generated from the input to the standard corpus excerpts. Define $q(x)$ as the query generated based upon the artifact; define the top-k k retrieval set Rk(x) as follows:

$$R_k(x) = \mathrm{TopK}_{d_i \in D}\ s(q(x), d_i) \tag{1}$$

Where $s(q(x), d_i)$ measures similarity between the artifact-derived query and each excerpt in the standards corpus. If using vector representations to compute similarity, then cosine similarity can be used as follows:

$$s(q(x), d_i) = \frac{\phi(q(x)) \cdot \phi(d_i)}{\|\phi(q(x))\|\, \|\phi(d_i)\|} \tag{2}$$

where $\phi(\cdot)$ represents the norm. This is where the text input is transformed into an embedding space. The top-k snippets are appended to the prompt so that the model will produce audit reports conditioned on technical evidence and appropriate standards. The output produced by this system will consist of short-form audit reports, each consisting of three parts: the security finding that was discovered; the mapped security control or principle that corresponds to the finding; a proposed remedy for the finding. These parts of the report can be represented collectively as $y = (f, c, r)$. While generating these parts of the report may include generating readable text, it also involves creating a practical security assessment from the parts of the report collectively. Generation occurs within an autoregressive model, specifically:

$$P(y \mid x, R(x)) = \prod_{t=1}^{T} P(y_t \mid y_{1:t-1}, x, R(x)) \tag{3}$$

where y_t denotes the token generated at time t. Therefore, although the retrieval layer provides different contexts during generation than were provided when training the model, the basic architecture of the model remains unchanged. A manually created $\mathcal{X} = \{x_1, x_2, \ldots, x_N\}$ of security-related configuration files has been created. Each file contains specific patterns known to be either secure or unsecure. Examples of unsecure patterns included in the dataset are examples of running code with elevated privileges, setting root user access, use of insecure image tags, and weak isolation configurations. In addition to configuration files, there is also a list of expected observations associated with each file, which serve as reference labels for a manual review.

A. Dataset Selection

This is an evaluated data base with a collection of open source repositories for which the most common applications

are used in several different technical areas. Examples of these repositories would include cloud computing technology including Kubernetes, Docker, Terraform, and Helm. Other examples of repositories would include those related to security and networking technologies, i.e., OpenSSL, Vault, Trivy, Cilium, Open Policy Agent, and Cosign. Repositories related to database and message queueing technologies also exist in this data base, e.g., PostgreSQL, MySQL, Redis, Elasticsearch, and Kafka. Web framework repositories have been included as well in order to test the system's capabilities using both modern and legacy development technologies.

The use of this wide variety of repositories will enable testing of the system using a variety of programming languages, architectures, and configurations. By doing so, it provides a basis for understanding whether or not the system can operate effectively outside of a specific context. Additionally, through the use of complex, large scale projects, i.e., Kubernetes and Node.js, it will allow for the testing of how well the system can handle large code bases while maintaining contextual integrity.

Finally, since the repository documentation is complete and has been thoroughly documented, the RAG retrieval-augmented generation method can be evaluated based upon whether or not the system can retrieve relevant security control information and apply it appropriately. Furthermore, because only mature, widely-reviewed repositories were selected for inclusion in the data base, a solid foundation exists for evaluating the results obtained from using this data base.

This research compares 3 methods under similar levels of prompts as follows: (1) A Direct Baseline Using a General-Purpose Language Model Without Retrieval; (2) The original version of the OpenCode Agent; (3) An Enhanced Version of the OpenCode Agent Using a Retrieval Augmented Generator (RAG). The only systematic differences between these systems was that all were provided the exact same Audit Instructions and the Proposed Method was supplemented with Retrieved ISO Control Excerpts.

Evaluations will be based on an evaluation Rubric. For each Sample x_i and for each Model M_j, there will be 3 criteria evaluated: Finding Correctness $s_{ij}^{(F)}$, Control Relevance $s_{ij}^{(C)}$, and Recommendation Usefulness $s_{ij}^{(R)}$; Each criterion will be rated on a scale of 0-2. Thus, for each Model-Sample Pair the Total Score would be

$$T_{ij} = s_{ij}^{(F)} + s_{ij}^{(C)} + s_{ij}^{(R)} \tag{4}$$

with $0 \leq T_{ij} \leq 6$. The Average Performance Across the Dataset would then be Calculated as

$$\bar{T}_j = \frac{1}{N} \sum_{i=1}^{N} T_{ij} \tag{5}$$

The analysis of criterion-specific averages is conducted in order to identify if use of retrieval improves specific performance criteria including how well performance aligns with standards.

Overall, the methodological approach allows for a feasible comparison of the effectiveness of using this method as part of a small-scale deployment while at the same time allowing for strong analytic capabilities. The method combines conditional text generation, retrieval of standards from very few examples, and manual evaluation of each report into one common comparative method. The ability to conduct a practical analysis on whether or not the incorporation of retrieval-augmented generation will allow for better production of security audit reports by an open source coding agent.

VI. MODEL WORKFLOW

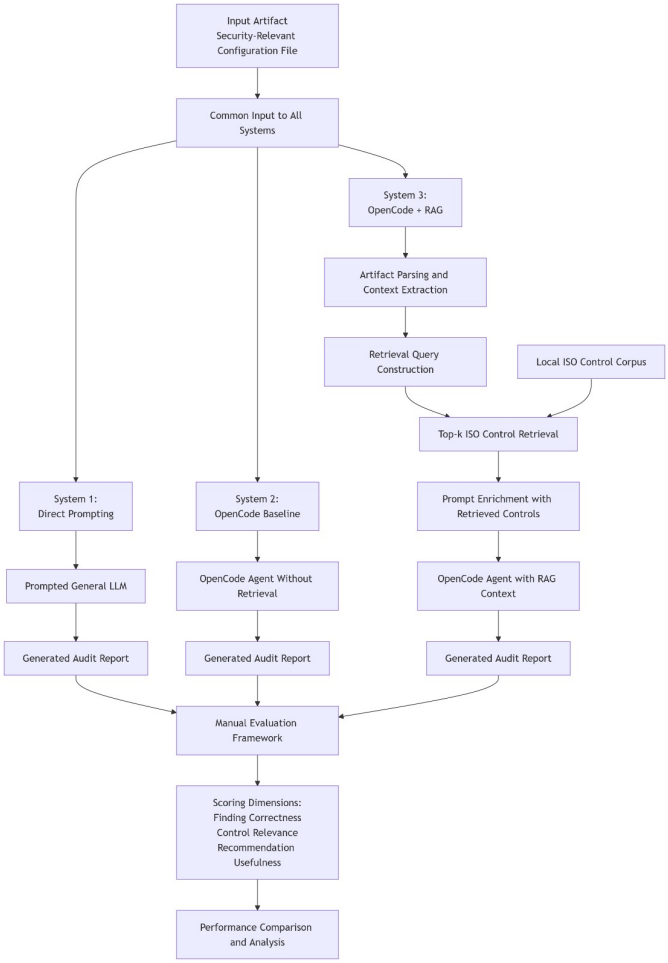

Fig. 1. Flowchart

As shown in Fig.1, this system employs a multi-stage RAG (retrieval-augmented generation) process for automatically evaluating software repositories in comparison to the requirements of the ISO/IEC 27001:2022 information security management standard. Once the input path to the intended repository has been entered into the system, the ingestion module will traverse the file directory structure using a recursive approach. However, all non-semantic directories such as .git, node modules, dist, and .next are excluded from this traversal. In addition to other forms of raw textual content

extraction from various supported file formats (TypeScript, JavaScript, Python, Go, Rust, Java, Ruby, PHP, Terraform, SQL, shell script files, etc.), the ingestion module will also extract raw textual content from supported configuration file formats (YAML and TOML), markdown documentation files and dockerfile files. Raw textual content extracted from PDF files is accomplished through the use of the pdf-parse library. File contents that have been extracted by the ingestion module are then concatenated together along with an annotation that identifies their relative file location. All concatenated file contents are limited to a maximum of 120,000 characters. When a file's contents exceed 120,000 characters they are truncated. Truncations that occur due to exceeding the maximum allowed character limit are noted as a boolean "clipped" value in the run metadata associated with each generated output.

Simultaneously, an exhaustive knowledge base of security controls is built from three authoritative pdf documents based on ISO/IEC 27001. The contents of the documents are divided into non-overlapping segments of 1400 characters; the segment is then serialized as a SecurityControl object into a json file for storage. In rag mode, the system initiates the retrieve phase by projecting each control document into a 256 dimensional vector space with the deterministic FNV-1A hash projection. The resulting projections are inserted into one of two types of vector stores: local (pinecone, qdrant) or cloud-based. Following insertion, the system will obtain the k most similar control segments based upon the cosine similarity of their respective embeddings to the vector representation of the source code that was used to construct the initial projection.

Following the retrieval process, the control selection(s) and source code that were identified for ingestion are sent to the "Prompt Engineering Module" (PEM). PEM dynamically creates a structured audit prompt based on the data submitted. There are three different formats that PEM can generate audit prompts depending on whether you select direct, baseline or rag format. In direct mode, the content of your file is embedded directly into the prompt. In baseline mode, the content of your file will be included as an attachment but not part of the controls in the prompt. In RAG Mode, both your file content and the top k controls with their respective citations will be attached to your prompt. Once the structured audit prompt has been created it is dispatched to either a stateless inferencing engine for direct mode or a stateful open-code agentic session for baseline and RAG Modes. All resultant outputs from the assistive technology are then combined into a single raw report string. Subsequently this report string is analyzed using a heuristic verification module that evaluates your report against five criteria. These include: structural completeness; evidence grounding; control alignment; logical consistency; and generic reporting. The result of this analysis is a standardized verification score between [0.0, 1.0].

All generated artifacts (the audit report; retrieved controls; verification results; data about the artifacts (metadata); and the entries from the run manifest) will be written to an output directory with timestamps. This allows us to build on our datasets over time as long as the same benchmark is executed multiple times.

VII. IMPLEMENTATION

A. System Architecture and Runtime

OpenCode is an open-source, software-based application that has been developed by utilizing the modular monorepo design model. In this particular design model, the primary runtime logic is contained within the core package and all other auxiliary packages contain UI's, integration features, etc. The OpenCode System utilizes the Bun JavaScript Runtime environment as well as Type Script source files. Additionally, the OpenCode System uses schema driven definitions (Zod) to define validation for configurations and contracts for APIs. These definitions enable both static type checks as well as runtime validation at the point of integration.

The Agent's Inference Loop uses real-time language model output in combination with Session Management to manage the interactions with the Model. Interactions occur using the Vercel AI SDK that generates token and event streams. Unlike traditional completion APIs these streams are used for processing in a session based upon the Effects Library. The Effects Library provides features such as Structured Concurrency, Explicit Resource Lifetime Management (i.e., scoped abort signal) and Typed Service Composition. As a result it is able to provide better and more maintainable control flow then using traditional Promise Chains.

Unlike the traditional Chat Transcript each interaction Turn is represented as a Graph containing the User Input, Assistant Responses and Tool Outputs. By representing each Turn this way we are able to track correlations amongst the multiple tool calls invoked by the user, their respective outputs, reasoning used and any textual output from the tools.

B. Tooling, Protocols, and Governance

The interface to the language model is through an object registry that includes both static core primitives (e.g., file I/O, search, shell commands, etc.) and dynamic primitives which are found by the system. The system implements the MCP for connecting other tools and resources through standardized communication mechanisms such as stdio and http. In cases where authentication is needed (OAuth), it will be handled in this manner. The structure of the system enables the separation of local execution on behalf of the agent versus remote/standalone tool executions while enabling extensionability and maintaining a clear security boundary.

A permissions layer governs operations that could potentially have sensitive actions. The layer uses declarative (permission) rules that use wildcard patterns of action resource names to make decisions about how an operation should be handled (e.g., allow, deny, or prompt). The permissions layer is in addition to "agent profile" which defines the operational mode for an agent with restrictions on what types of actions can be taken (e.g., write or execute commands), allowing the same code base to work at multiple levels of assurance (i.e., ranging from exploration analysis to fully integrated development workflows.)

C. LLM Integration and Control Infrastructure

The Core allows for integration of multiple Large Language Model (LLM) Providers through a single abstraction layer which can be used to access either Commercial or Open-Weighted Endpoints. There are compatibility layers and shims that provide for differences in each of the LLM Vendor's APIs. Each of the authentication methods is Provider Specific. This provides an ability to select from API Keys, Oauth, Cloud Credential Chains without having to modify the Session Logic. In order to support integration with Editors/IDEs, the system will implement protocols such as the Agent Client Protocol (ACP). The ACP protocol will expose session management and prompt operations to client applications outside of the primary Terminal Interface.

The Control Plane for this Model is a REST API that utilizes Hono. It uses a simple RESTful endpoint to manage Workspace Sessions Configuration Events Streams. Since there are many high frequency events being sent to the Control Plane in real-time (streaming), we have selectively disabled compression to improve Latency. We have also added operational monitoring by including both Request Logging and Structured Timing Instrumentation. The Server Push Architecture for the Control Plane matches well with the Streaming Output Characteristics of the Model. This allows us to utilize WebSockets and Server-Sent Event Channels for asynchronous notifications.

D. Persistence, Interfaces, and Extensions

Persistent data (session metadata, etc.) as well as associated relational data models are stored in a local Drizzle ORM database. The schema for this database will be migrated based upon declarations contained within modules. Because these module declarations can also provide type information in TypeScript, they ensure that the schema of your project evolves in a predictable manner consistent with that which would exist if you were working directly in TypeScript. The use of separate, project specific, file-based storage systems for each type of state (e.g., ignore files and their contents; execution snapshots; workspace-related metadata) allows each system to manage its own state such that side effects of the tools that run against them can be easily distinguished from one another both at the level of sessions and directories.

The common base of interaction with users is provided through many different interfaces using the same architecture; e.g., command-line (terminal) interface(s), an optional set of desktop applications, and/or a web or console application that can be run from separate repository(ies). The same code is accessed programmatically by an API (JavaScript SDK) to support integration/automated testing and distribution. In addition to these interfaces, there are documented examples of experimental methodologies (e.g., security audit mode that utilizes control corpora augmented by retrieval information, and/or a vector backend) which may be referenced based on relevance to research contributions.

E. Engineering Constraints and Trade-offs

Design choices regarding type-safe interface, explicit cancellation of tasks, and hooking into plugins that allow researchers to add new or "experimental" features have resulted in an expanded set of dependencies for this project (i.e., AI SDK, Effect, MCP, ORM, and UI). While such trade-offs are typical for many research projects which seek to combine the need for maintainable code with the flexibility necessary to support the rapid development of agents' policies and tools.

VIII. RESULTS

A. Overall Performance Analysis

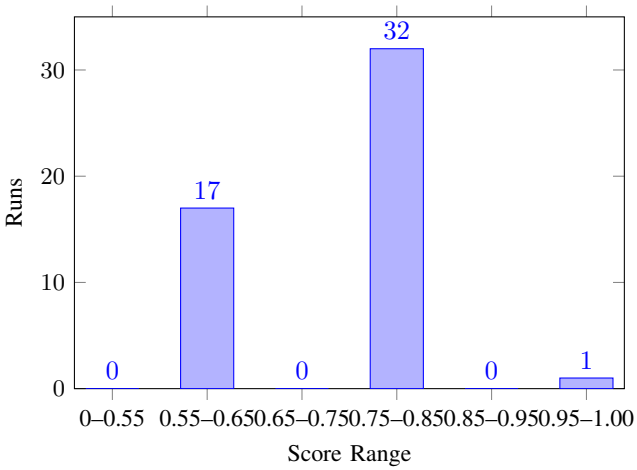

Fig. 2. RAG Headline Score Distribution

An assessment of the RAG-enabled OpenCode system has been conducted to evaluate its performance in accordance with an open source benchmark comprised of 50 repository level audit task descriptions. A total of five evaluation criteria were applied to each output to generate a normalized score for each output in the interval [0,1] for use in assessing the quality of each output. On average the headlines for the OpenCode system scored a 0.736 with a median value of 0.800 and a standard deviation of .103. Of the 50 test cases performed only 2% resulted in a "full pass" as defined by achieving the maximum possible score of 5 points. Thus, although the results are consistent with other studies [7]; it appears that there will continue to be challenges associated with fully verifying compliance with the specified constraints. Fig. 2 presents a graphical representation of the empirical distributions.

Fig. 2 indicates that 98% of the produced OpenCode system outputs fell into a score range from 0.60 to 0.80. Therefore, this suggests that approximately 90-95% of the produced OpenCode system outputs failed at least one check per evaluation cycle and consequently failed to achieve complete compliance with the specifications. These results suggest that although the system generates well organized and understandable audit report(s) related to repositories in question, it fails to comply completely with all applicable constraints.

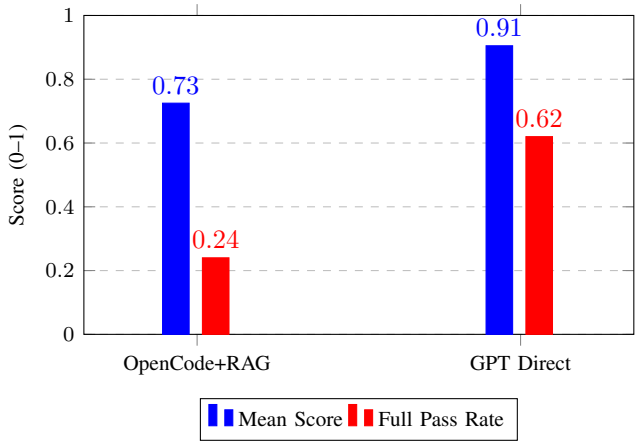

Fig. 3. Comparison of OpenCode + RAG and direct GPT prompting across shared evaluation metrics

B. Comparative Evaluation with Baselines

To assess the effectiveness of retrieval augmentation, the system was compared against a direct prompting baseline. On the shared four evaluation criteria (excluding control grounding), the RAG system achieved a mean score of 0.725, while the direct GPT-based system achieved a higher mean of 0.905 with a 62.0% full-pass rate. A comparative overview of these results is presented in Fig. 3.

Although the results appear to favor one model over another; however, several critical factors that affect experimental design are confounding variables. These include a difference in model architectures, a smaller number of input data and the fact that retrieval was not used as a constraint in the control. Although RAG models process much larger multi-file input, often resulting in truncated models (and therefore an increase in both structural/parsing complexity), the comparison shows differences in testing conditions versus pure ability of each model.

C. Check-wise Performance Analysis

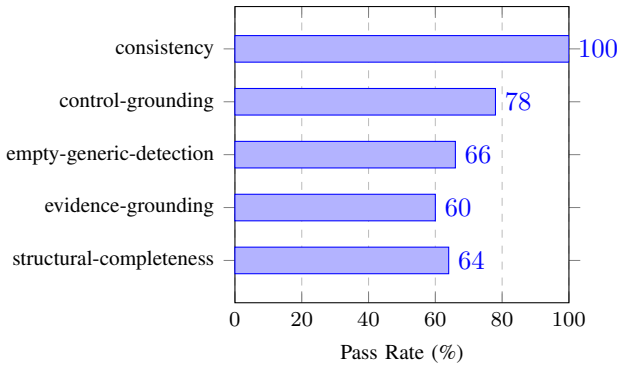

Fig. 4. Check-wise pass rates of the RAG-enabled OpenCode system across evaluation criteria

The data distribution of evaluation results for all criteria are shown in figure 4. Consistency (100%) has been the highest performance criteria as it measures how logical the system's responses were. The lowest passing score for Evaluation Criteria was the Evidence Grounding Criterion at 60%. Thus, the greatest weaknesses of the system have been within the Evidence Grounding criterion. Completeness of Structural Components and Generic Content Detection both had success rates of 64% and 66%, respectively. The Control-Grounding Metric, that is the RAG Framework's metric specific to this application, produced an overall success rate of 78% demonstrating the systems ability to link retrieved ISO Control References to its generated output.

D. Distributional and Statistical Insights

TABLE I
PERCENTILE-BASED PERFORMANCE COMPARISON

Percentile	OpenCode + RAG (÷5)	GPT direct (÷4)
p10	0.600	0.750
p25	0.600	0.750
p50	0.800	1.000
p75	0.800	1.000
p90	0.800	1.000

Percentile-based analysis further confirms the discrete nature of evaluation scores, as shown in Table I. For the RAG system, the median (p50) is 0.800, while lower percentiles (p10, p25) remain at 0.600, indicating a left-tailed distribution. This suggests that a subset of runs underperform due to failure in specific checks, primarily evidence grounding.

In contrast, the baseline system demonstrates a higher concentration of perfect scores, with p50, p75, and p90 all reaching 1.000 on the shared evaluation scale. However, this reflects reduced evaluation complexity and input size rather than inherently superior reasoning capability.

E. Key Observations and Implications

The Results show that although Direct Prompting does better on numeric scores when the problem is a simple one, RAG has major advantages (traceability, explainability) for aligning with ISO Standards and Auditing. Therefore, control grounding, which will ensure that the output from the Audit are both syntactically correct and semantically aligned to the authoritative ISO Guidelines, was included in this study.

Ultimately, our findings provide evidence to support our Hypothesis that Retrieval-Augmented Systems improve the "practical" ability to apply an LLM-based Security Audit System into real world environments where auditability and compliance are required.

IX. LIMITATIONS AND FUTURE SCOPE

The limited nature of this project's prototype implementation as well as the smaller scale and manual data set used in this project will limit the generalizability of the findings to other Enterprise environments. Additionally, there are no formal methods available to verify the accuracy of the output from the program that generates these artifacts. Therefore

future work needs to extend the number of documents in the repository of standards, include additional artifact types, automate some or all of the validation process for those generated artifacts, and conduct larger scale expert led assessments to understand the degree to which an enterprise can implement and support the use of this tool.

X. CONCLUSION

This paper presents a systematic but simplified methodology for automatically conducting security auditing through extending an open source coding agent using Retrieval-Augmented Generation (RAG) based on the ISO security control framework. A comparative assessment of RAG, direct prompt, and the baseline OpenCode system demonstrated that the inclusion of standard-based knowledge retrieval contributed to more accurate, better-structured, and more actionable audit reports. Although this research is still in its early stages, it demonstrates how combining technical configuration assessment with coding agents and knowledge retrieval could provide scalable cybersecurity support while also satisfying regulatory policy [1], [9].

REFERENCES

[1] H. Xu *et al.*, "Large language models for cybersecurity: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, 2024.

[2] X. Cao *et al.*, "Advancing LLM-based security automation with customized group relative policy optimization for zero-touch networks," *IEEE J. Sel. Areas Commun.*, 2025.

[3] P. Lingga, J. Jeong, J. Yang, and J. Kim, "SPT: Security policy translator for network security functions in cloud-based security services," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5156–5169, 2024.

[4] S. Lee and J. Nam, "Kunerva: Automated network policy discovery framework for containers," *IEEE Access*, vol. 11, pp. 95616–95631, 2023.

[5] D. Saha *et al.*, "LLM for SoC security: A paradigm shift," *IEEE Access*, vol. 12, pp. 155498–155521, 2024.

[6] W. Wang *et al.*, "NLACPs dataset expansion with AI: Leveraging ChatGPT for policy generation," *IEEE Internet Things J.*, 2026.

[7] L. Ofusori, T. Bokaba, and S. Mhlongo, "Artificial intelligence in cybersecurity: A comprehensive review and future direction," *Applied Artificial Intelligence*, vol. 38, no. 1, p. 2439609, 2024.

[8] M. A. M. Farzaan, M. C. Ghanem, A. El-Hajjar, and D. N. Ratnayake, "AI-powered system for an efficient and effective cyber incidents detection and response in cloud environments," *IEEE Trans. Mach. Learn. Commun. Netw.*, 2025.

[9] S. Sai, U. Yashvardhan, V. Chamola, and B. Sikdar, "Generative AI for cyber security: Analyzing the potential of ChatGPT, DALL-E, and other models for enhancing the security space," *IEEE Access*, vol. 12, pp. 53497–53516, 2024.

[10] A. Schreiber and I. Schreiber, "AI for cyber-security risk: harnessing AI for automatic generation of company-specific cybersecurity risk profiles," *Inf. Comput. Secur.*, vol. 33, no. 4, pp. 520–546, 2025.